16-833: Robot Localization and Mapping, Spring 2026

Homework 3 - Linear and Nonlinear SLAM Solvers

1 Measurement function (10 points)

Given robot poses $\mathbf{r}^t = [r_x^t, r_y^t]^\top$ and $\mathbf{r}^{t+1} = [r_x^{t+1}, r_y^{t+1}]^\top$ at time t and $t+1$, write out the measurement function and its Jacobian. (5 points)

$$h_o(\mathbf{r}^t,\ \mathbf{r}^{t+1}) : \mathbb{R}^4 \to \mathbb{R}^2,$$
$$H_o(\mathbf{r}^t,\ \mathbf{r}^{t+1}) : \mathbb{R}^4 \to \mathbb{R}^{2\times 4}.$$

Similarly, given the robot pose $\mathbf{r}^t = [r_x^t,\ r_y^t]^\top$ at time t and the k-th landmark $\mathbf{l}^k = [l_x^k,\ l_y^k]^\top$. (5 points)

$$h_l(\mathbf{r}^t, \mathbf{l}^k) : \mathbb{R}^4 \to \mathbb{R}^2,$$
$$H_l(\mathbf{r}^t, \mathbf{l}^k) : \mathbb{R}^4 \to \mathbb{R}^{2\times 4}.$$

1.1 Answer

Odometry Measurement:

Given two consecutive robot poses:

$$\mathbf{r}^t = \begin{bmatrix} r_x^t \\ r_y^t \end{bmatrix}, \quad \mathbf{r}^{t+1} = \begin{bmatrix} r_x^{t+1} \\ r_y^{t+1} \end{bmatrix}$$

The odometry measurement represents the relative displacement:

$$h_o(\mathbf{r}^t, \mathbf{r}^{t+1}) = \begin{bmatrix} r_x^{t+1} - r_x^t \\ r_y^{t+1} - r_y^t \end{bmatrix}$$

The Jacobian is:

$$H_o = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix}$$

Landmark Measurement:

Given robot pose and landmark:

$$\mathbf{l}^k = \begin{bmatrix} l_x^k \\ l_y^k \end{bmatrix}$$

The measurement is:

$$h_l(\mathbf{r}^t, \mathbf{l}^k) = \begin{bmatrix} l_x^k - r_x^t \\ l_y^k - r_y^t \end{bmatrix}$$

The Jacobian is:

$$H_l = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix}$$

since both measurement functions are linear, so their Jacobians are constant.

1.1 Plots for 1.4

1. Now proceed with `2d_linear.npz`, visualize the trajectory and landmarks, and report the efficiency of your method in terms of run time. Attach the visualization and analysis of corresponding factor for `qr`, `qr_colamd`, `lu`, `lu_colamd`. What are your observations and their potential reasons (general comparison between QR, LU, and their reordered version)? (10 points)

Table 1: 2d_linear.npz

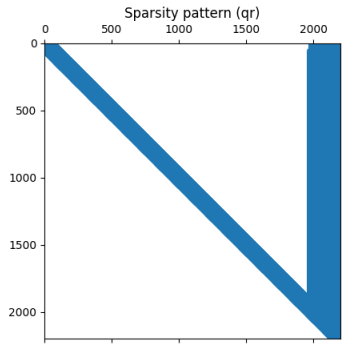
Method	**Average Time (s)**
QR	0.321
QR_COLAMD	0.175
LU	0.027
LU_COLAMD	0.041

The trajectories and landmarks from all four methods (QR, QR_COLAMD, LU, and LU_COLAMD) are similar and closely match the ground truth, which shows that all methods are solving the system correctly. But, there is a clear difference in runtime. From the table, LU is the fastest method, followed by LU_COLAMD, then QR_COLAMD, and finally QR being the slowest. This is expected because LU solves the normal equations, which are easier to compute, while QR works directly on the original system and is more computationally expensive.

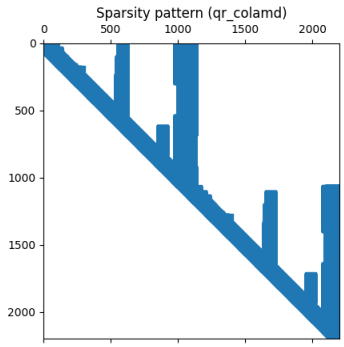
When comparing the reordered versions, QR_COLAMD is significantly faster than QR. This is because COLAMD reorders the matrix to reduce fill-in during factorization, making the computation more efficient. A similar effect is seen for LU, although the improvement is smaller in this case. From the sparsity plots, we can see that COLAMD produces a cleaner and more sparse triangular matrix compared to the natural ordering.

Overall, QR is more numerically stable, while LU is faster but can be slightly less stable since it works on the normal equations. The COLAMD ordering improves efficiency for both methods by exploiting sparsity in the system.

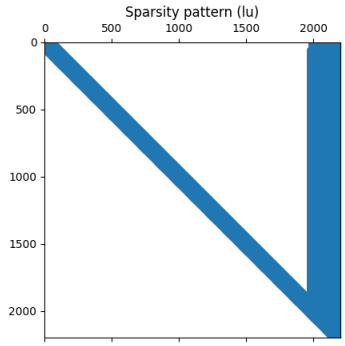
The plots:



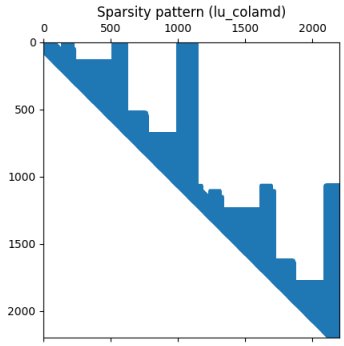
(a) QR



(b) QR_COLAMD

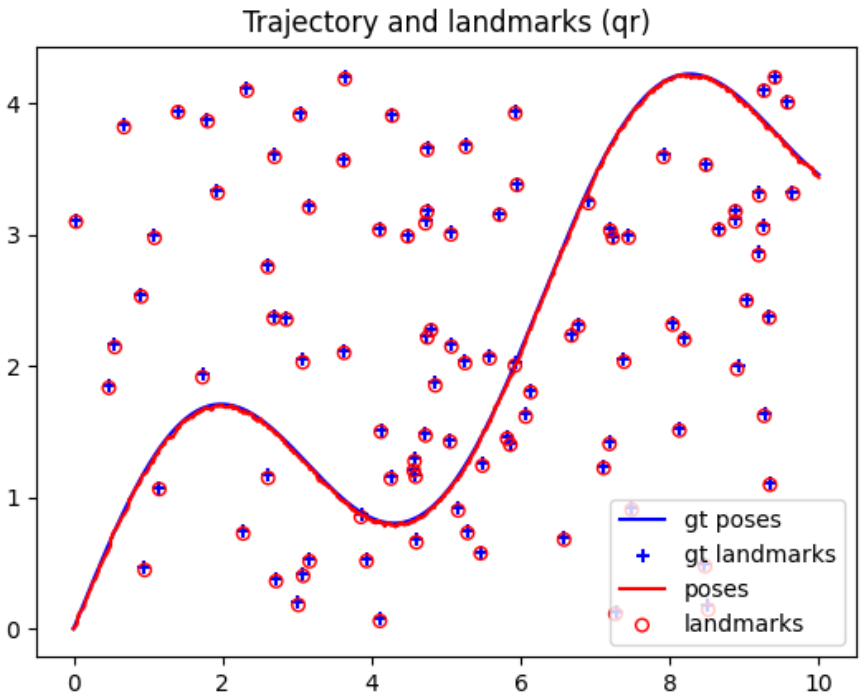


(a) LU

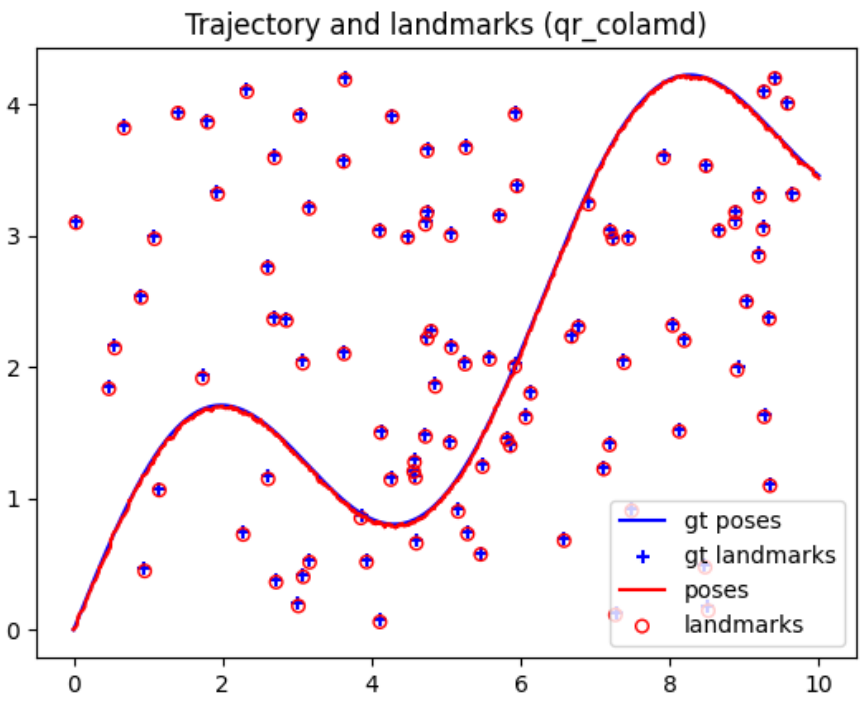


(b) LU_COLAMD

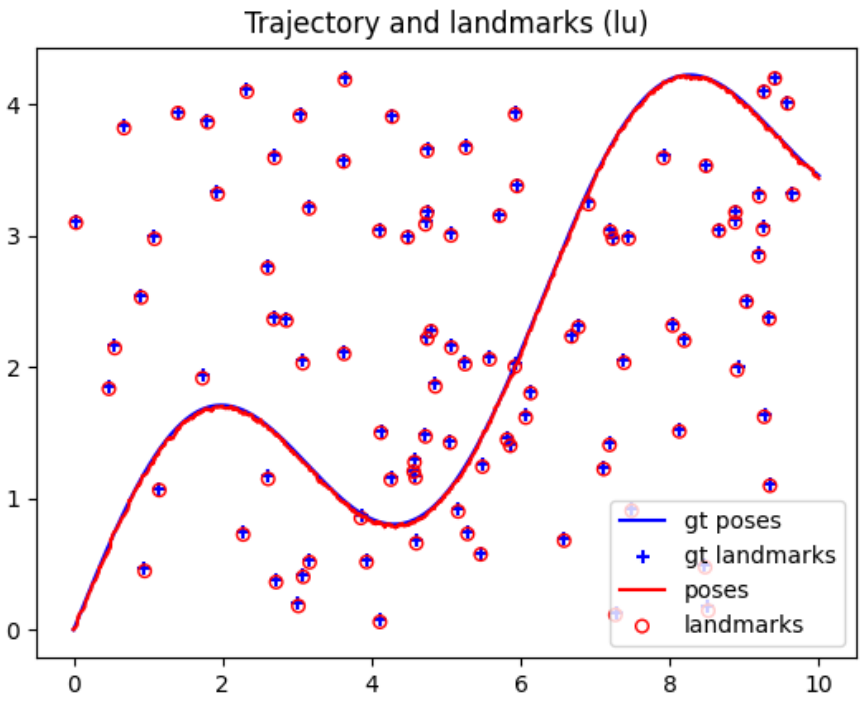
The trajectory and landmark plots are:



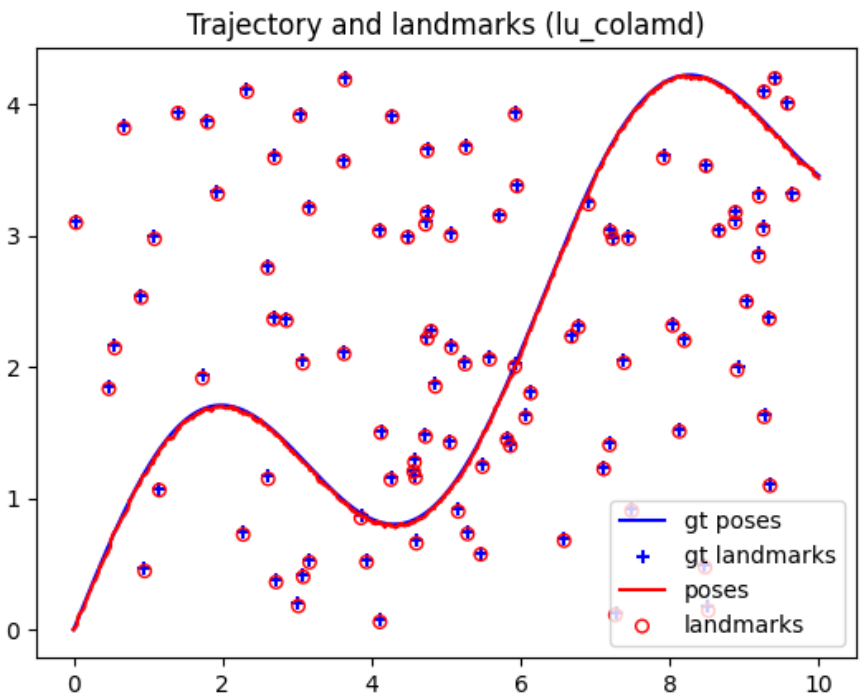
(a) QR



(b) QR_COLAMD



(c) LU



(d) LU_COLAMD

2. Similarly, process `2d_linear_loop.npz`. Are there differences in efficiency comparing to `2d_linear.npz`? Write down your observations and reasoning. (10 points)

Table 2: 2d_linear_loop.npz

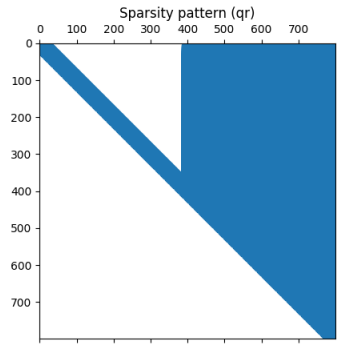
Method	Average Time (s)
QR	0.111
QR_COLAMD	0.014
LU	0.021
LU_COLAMD	0.006

For the 2d_linear_loop.npz dataset, we observe that the differences in efficiency between the methods are more significant compared to the 2d_linear.npz case. In particular, the COLAMD versions (QR_COLAMD and LU_COLAMD) are much faster than their corresponding non-reordered versions. For example, QR_COLAMD is significantly faster than QR, and LU_COLAMD is the fastest overall.

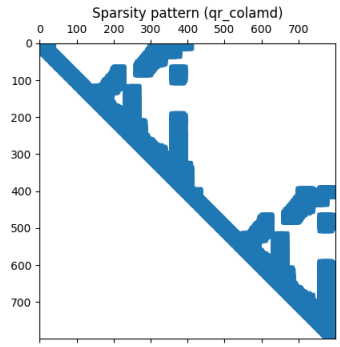
This happens I feel because the loop closure introduces additional constraints, which makes the system more densely connected. So there will be more fill-in during matrix factorization. Without reordering, this increases computational cost. COLAMD helps reduce this.

Compared to the previous dataset, the benefit of COLAMD is much more noticeable here. QR remains slower but more stable, while LU is faster but slightly less stable. Overall, exploiting sparsity through reordering becomes more important in the loop dataset due to the increased complexity of the system.

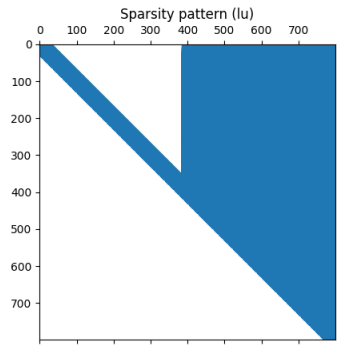
The triangular matrices visualizations:



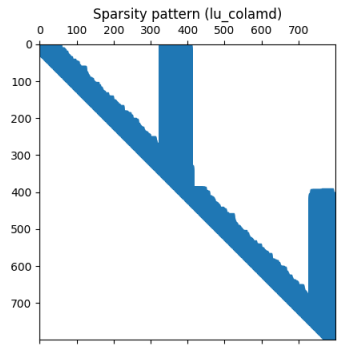
(a) QR



(b) QR_COLAMD

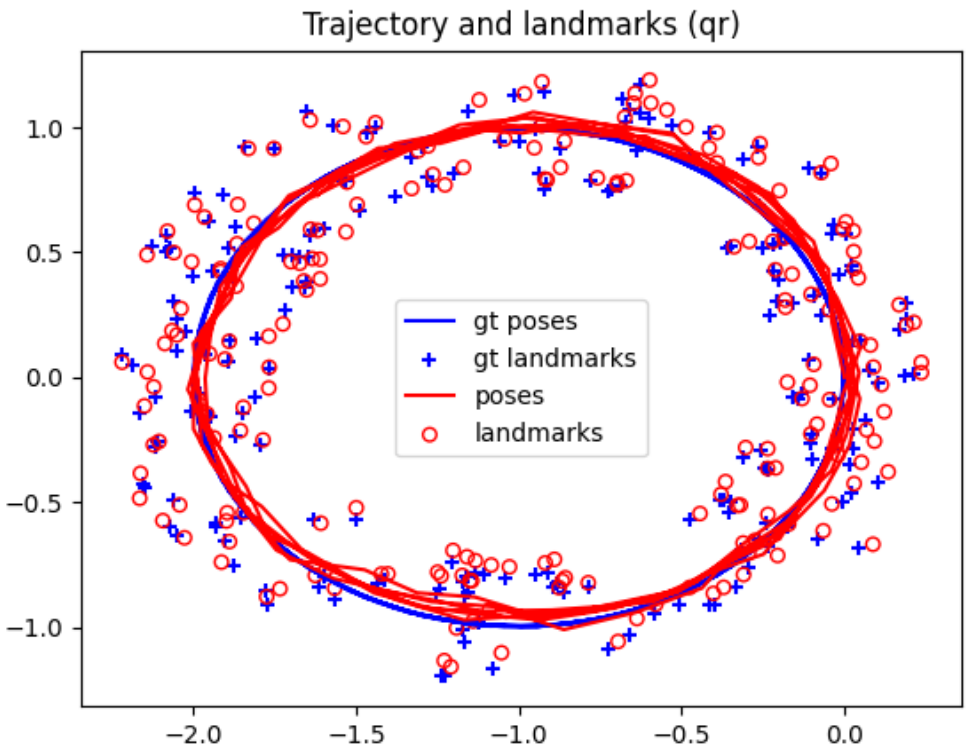


(a) LU

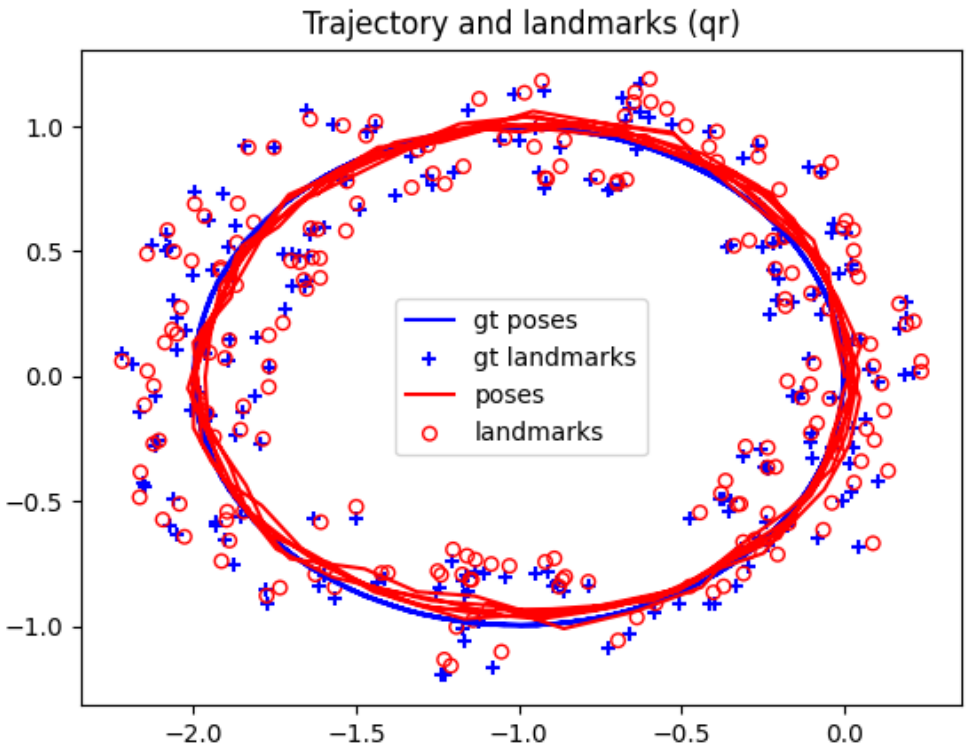


(b) LU_COLAMD

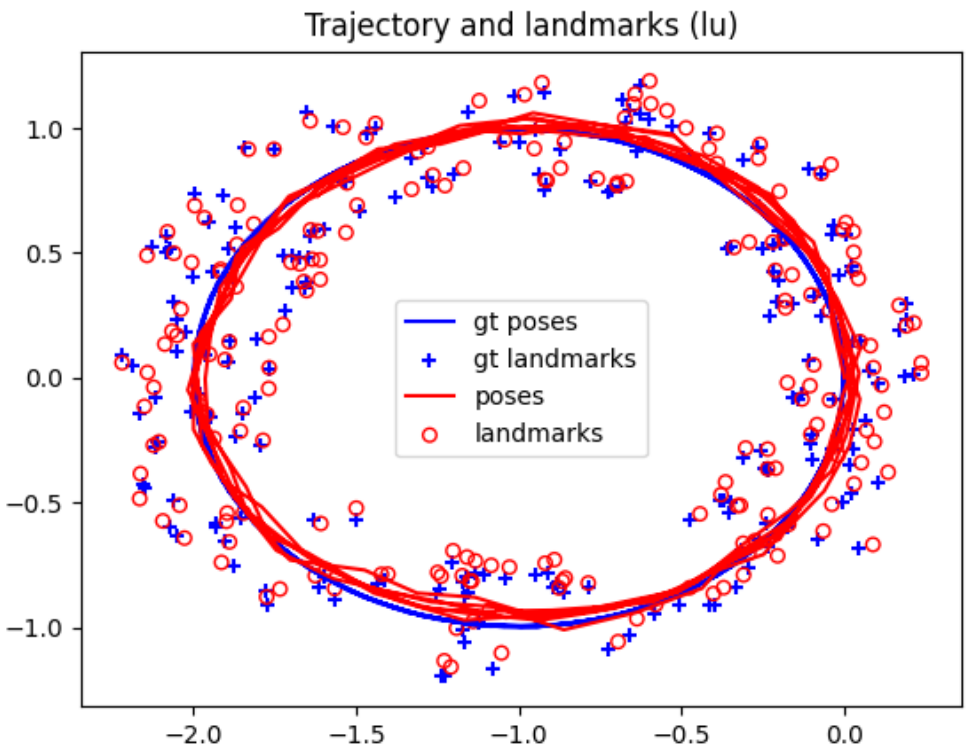
The trajectory and landmark plots are:



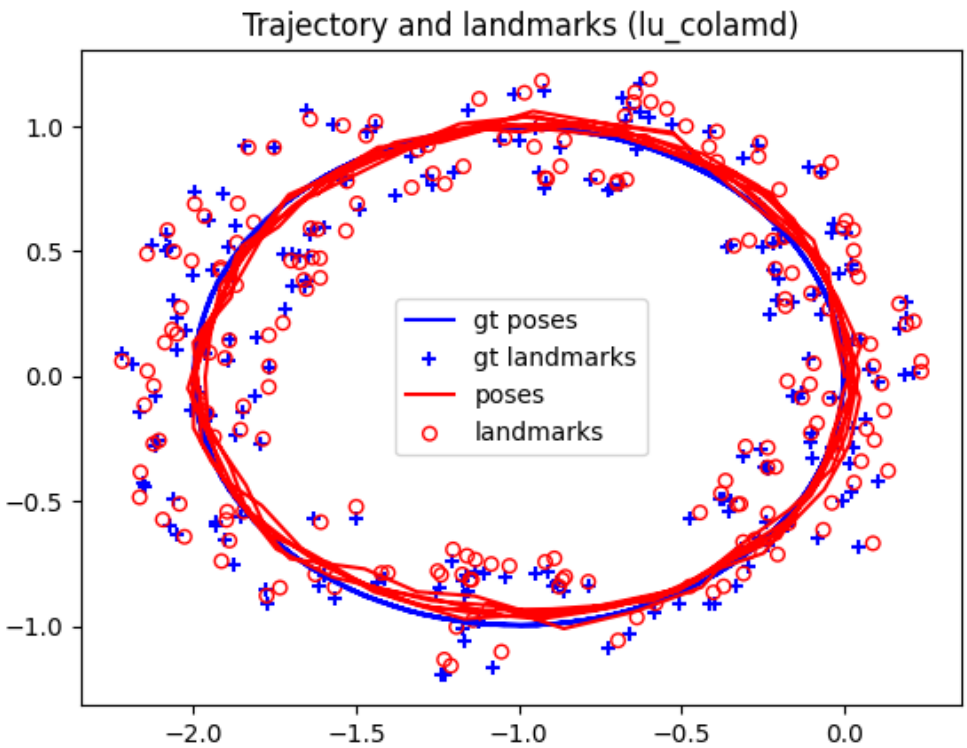
(a) QR



(b) **QR_COLAMD**



(c) **LU**



(d) **LU_COLAMD**

2 2D Nonlinear SLAM

2.1 Answer

From the eqaution 2 given the jacobian wrt X will be:

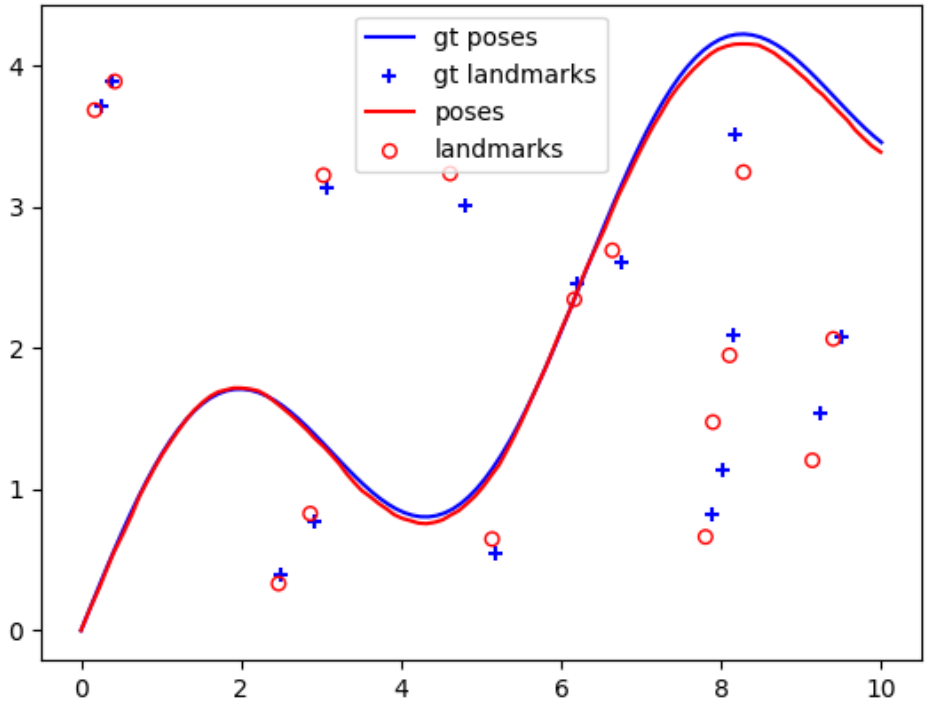
$$H_l(\mathbf{r}^t, \mathbf{l}^k) = \begin{bmatrix} \frac{\partial\theta}{\partial r_x^t} & \frac{\partial\theta}{\partial r_y^t} & \frac{\partial\theta}{\partial l_x^k} & \frac{\partial\theta}{\partial l_y^k} \\ \frac{\partial d}{\partial r_x^t} & \frac{\partial d}{\partial r_y^t} & \frac{\partial d}{\partial l_x^k} & \frac{\partial d}{\partial l_y^k} \end{bmatrix} = \begin{bmatrix} \frac{(l_y^k - r_y^t)}{q} & \frac{-(l_x^k - r_x^t)}{q} & \frac{-(l_y^k - r_y^t)}{q} & \frac{(l_x^k - r_x^t)}{q} \\ \frac{-(l_x^k - r_x^t)}{d} & \frac{-(l_y^k - r_y^t)}{d} & \frac{(l_x^k - r_x^t)}{d} & \frac{(l_y^k - r_y^t)}{d} \end{bmatrix}$$

where

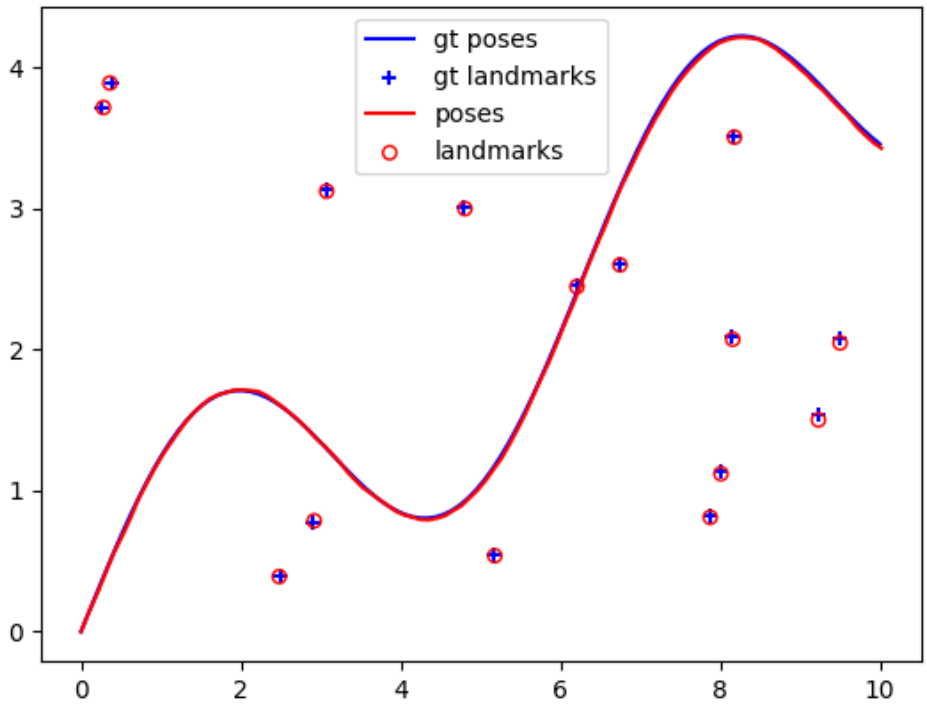
$$q = (l_x^k - r_x^t)^2 + (l_y^k - r_y^t)^2, \quad d = \sqrt{q}$$

2.3 Solver plots

The trajectory and landmark plots for **QR** are:



(a) Before optimization QR curve



(b) After optimization QR curve

The initial trajectory and landmarks before optimization are noisy and not well aligned with the ground truth (just one point has correspondence). This is be-

cause the initialization is obtained by directly chaining odometry and using raw observations, which accumulates error over time. After optimization, the trajectory becomes much smoother and the landmarks align closely with the ground truth.

Difference between Linear and Non Linear SLAM

The main difference between linear and nonlinear SLAM is that linear SLAM solves the system in a single step, while nonlinear SLAM requires iterative optimization. In the nonlinear case, the measurement model is not linear, so we linearize the system around the current estimate and solve it repeatedly using methods like Gauss-Newton.

At each iteration, the estimate improves until convergence. This makes nonlinear SLAM more computationally expensive but necessary for handling realistic measurement models such as bearing and range.The initial trajectory and landmarks before optimization are noisy and not well aligned with the ground truth. This is because the initialization is obtained by directly chaining odometry and using raw observations, which accumulates error over time. After optimization, the trajectory becomes much smoother and the landmarks align closely with the ground truth.